

■ Learning System

Architecture & Debugging Guide for Non-Coders

Understand how the system is built, how data flows,
and how to identify errors yourself.

Repository: [prashant08062-arch/learning-system](https://github.com/prashant08062-arch/learning-system)

Date: October 2026

1. The Big Picture (What Is This System?)

The Learning System is a **self-contained website** — a collection of HTML, CSS, and JavaScript files that run entirely in your web browser. There is no server, no database, no cloud. Everything the student sees is loaded from files on disk and rendered by the browser.

Think of it like a digital textbook with interactive animations. When a student opens `index.html` in their browser, the browser loads all the scripts, reads the chapter data, and renders the lecture, notes, practice problems, and self-test questions on screen. The student's progress (which chapters they've visited, which problems they've answered) is saved in the browser's **localStorage** — a small built-in storage area that persists between sessions.

The system has 3 layers:

Layer	What it is	Analogy
Data Layer	<code>chapter.js</code> files — the "content" (narration text, SVG diagrams, questions)	The pages of the book
Logic Layer	<code>app.js</code> , <code>tts.js</code> , <code>proctor.js</code> , etc. — the "engine" that reads the content and renders it	The teacher's brain and eyes
Presentation Layer	<code>index.html</code> + <code>style.css</code> — the "skin" (buttons, tabs, colors, fonts)	The book's cover and page design

Key insight: If something looks wrong on screen (blank area, missing text, frozen animation), the problem is in ONE of these three layers. This guide will help you figure out which layer is broken and where to look.

2. File-by-File Tour (What Each File Does)

Here is every file in the system, explained in plain English. You don't need to read the code — just understand what each file's **job** is.

■ HTML Files (the page structure)

File	What it does	If it breaks, you see...
index.html	The main page. Loads all scripts, shows the home screen with subject pages and "page error" button.	Blank white page or "page error"
parent-dashboard.html	A separate page parents log into to see student progress, test results, and other things.	Results and/or other things
mirrors_lenses_3d_atlas.htm	Standalone 3D physics atlas (separate from chapters).	3D atlas doesn't open.
water_body_atlas.html	Standalone 3D water body atlas (separate from chapters).	3D atlas doesn't open.

■ CSS Files (the visual styling)

File	What it does	If it breaks, you see...
css/style.css	All the visual styling: colors, fonts, button shapes, tab layout, alignment, etc.	Unpleasant style, dark theme regression

■■ JavaScript Files (the engine — what makes things work)

File	What it does (plain English)	If it breaks, you see...
js/app.js	THE MAIN ENGINE. Reads chapter data, builds the 7 tabs (Nothing, Notes, Spelling, Real, Play, Games, Extras)	Nothing, Notes, Spelling, Real, Play, Games, Extras
js/tts.js	Text-to-Speech engine. Reads the narration text aloud, splits paragraphs into chunks (Chapters have 15 paragraphs)	Nothing, but it doesn't speak (Chapter has 15 paragraphs)
js/auth.js	Authentication. Handles student signup, login, logout, parent login, ignore cases, signouts, forgot password	Login, Ignore Cases, signouts, forgot password
js/proctor.js	Auto-proctoring. When enabled, detects tab switching, requests screen images via extensions, displays error	Session screen images violations displayed error
js/testpaper.js	Test paper generation. Creates 15-question randomized tests for Math chapters by titles (The test is 15 questions)	Test Paper test chapters by titles (The test is 15 questions)
js/enhancements.js	Extra features: progressive hints in guided practice, common nouns appearing in guided practice, common nouns appearing in guided practice	Nothing happens during lectures, none start
js/chalkboard-timeline.js	Syncs SVG animations with TTS narration. Reveals SVG elements specific to the current scene (The scene is 15 paragraphs)	SVG elements specific to the current scene (The scene is 15 paragraphs)
js/globe.js	3D Earth globe viewer for Geography chapters. Uses Three.js	Globe doesn't load. Black box where
data/catalog.js	The master registry. Lists all subjects and chapters. When you load chapter, chapters appear in the	Load chapter, chapters appear in the

■ Chapter Data Files (the content)

Each chapter lives in `data/<subject>/<chapter_slug>/chapter.js`. For example, the Light chapter is at `data/physics/light_mirrors_lenses/chapter.js`. Each chapter file defines a single JavaScript object called `window.CHAPTER_DATA` with these sections:

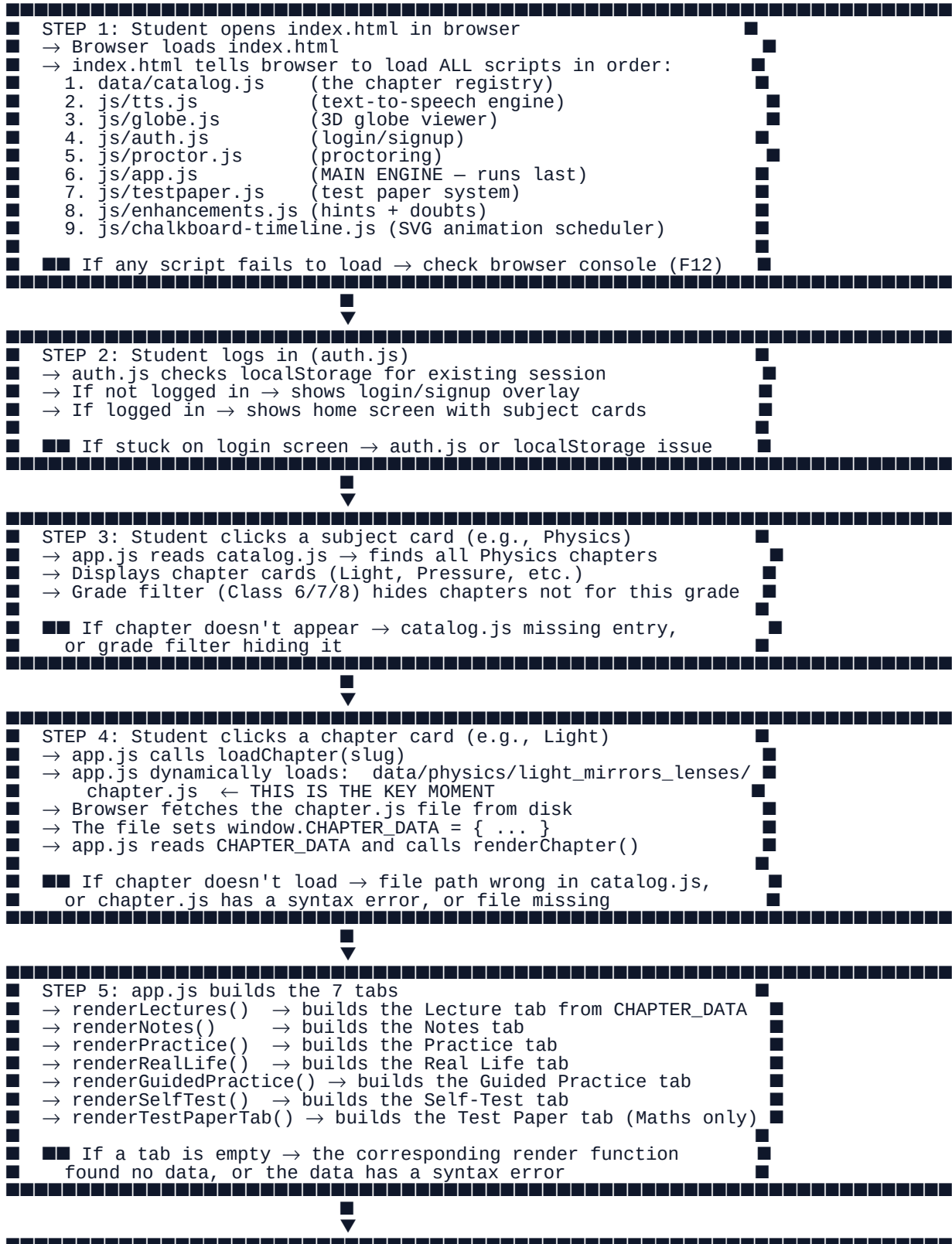
Section	What it contains	If it breaks, you see...
meta	Chapter identity: slug, title, subject, type (svg/image/gif/other), grade	Chapter doesn't appear in the right subject.

lectures	Array of lecture sections. Each has: id, label, viewBox, etc. (the diagram type). Canvas data is array of black lines b	Lecture tab is empty. Or SVG data is array of black lines b
notes	HTML string with key notes, vocabulary, formulas, memo tracks	Notes tab is empty or shows raw HTML code.
practice	HTML string with reveal-answer practice cards.	Practice tab is empty. Or "Show Answer" buttons don't work
realLife	Array of real-life scenarios, each with its own SVG and Reply to the scenario	RealLife tab is empty. Or scenarios don't advance.
guidedPractice	Array of interactive problems with step-by-step validation	GuidedPractice tab is empty. Or student can't see the
selfTest	Array of questions with answers. Student clicks "Show Answer" to reveal	SelfTest tab is empty. Or answers don't appear.

3. How Data Flows (From File to Screen)

Here is the step-by-step journey of how a chapter goes from a file on disk to the screen the student sees. Follow this flow to understand where things can break.

The Data Flow Pipeline



```

■ STEP 6: Student clicks ■ Play on a lecture
■ → app.js calls goToBeat(0)
■ → Shows beat 1's SVG group (<g class="el" data-beat="1">)
■ → Calls TTS.speak(beat text) → reads narration aloud
■ → Calls startChalkboardTimeline() → reveals SVG elements
■ progressively at specific timestamps
■ → When TTS finishes → advances to beat 2 → repeats
■
■ ■■ If screen is blank → SVG elements have opacity:0 and the
■ timeline didn't fire to set opacity:1
■ ■■ If elements are off-screen → viewBox doesn't match the
■ SVG coordinates (content is outside the visible area)
■ ■■ If audio leaks → TTS.speak called after tab switch/logout

```

4. The Rendering Pipeline (What Happens When You Click Play)

This is the most important section for debugging. When you click the ■ Play button on a lecture, a specific sequence of events happens. If ANY step fails, you see a blank or broken screen. Here's the sequence:

Step 1: `goToBeat(0)` is called

`app.js` sets the current beat to 0 (the first beat). It calls `state.updateUI()` which triggers `renderTranscript()` (shows the beat text in the right panel) and `renderBoard()` (shows the SVG diagram in the left panel).

Step 2: `renderBoard()` reveals SVG elements

This function finds all SVG elements with `class='el'` and sorts them by their `data-beat` value. For beat 0 (the first beat), it shows the first element by adding the 'visible' CSS class. The CSS rule `.el { opacity: 0 }` makes elements invisible by default; `.el.visible { opacity: 1 }` makes them visible. If this CSS rule is missing or overridden, elements stay invisible.

Step 3: `TTS.speak()` reads the narration

The text-to-speech engine reads the beat's narration text aloud. It splits long text into ~200-character chunks (Chrome's `speechSynthesis` has a 15-second bug). Each chunk is spoken sequentially. When all chunks finish, the `onEnd` callback fires.

Step 4: `startChalkboardTimeline()` reveals elements progressively

If the beat has a 'timelines' entry, `chalkboard-timeline.js` schedules `setTimeout` calls to reveal specific SVG elements at specific timestamps. For example, at `delay=4000ms`, it sets `element.style.opacity = '1'` for the mirror-glass rect. This creates the 'drawing itself' effect synced with the narration. If this doesn't fire, elements with `opacity:0` stay invisible.

Step 5: `onEnd` fires → advance to next beat

When TTS finishes, the `onEnd` callback calls `setTimeout(advance, 500)` — wait 500ms, then go to the next beat. `goToBeat(1)` repeats the cycle: reveal beat 2's SVG, speak beat 2's narration, schedule beat 2's timeline.

Step 6: Lecture ends at the last beat

When the last beat finishes, `state.isPlaying` is set to false and the play button resets to ■. The screen shows the final SVG state (all accumulated elements visible).

Where Things Go Wrong (The 3 Most Common Failures)

Failure	Symptom	Root Cause	Where to Look
---------	---------	------------	---------------

Blank screen on a beat	SVG area is dark/empty. No diagram visible.	SVG elements have opacity:0 and chalkboard timeline didn't fire to set opacity:1.	1. Check if the beat's <g class="el"> exists in chapter.js 2. Check if elements have style="opacity:1" (visible by default) or style="opacity:0" (needs timeline) 3. Check chalkboard-timeline.js is loaded
Content off-screen (clipped)	Only part of the diagram is visible. Text/labels cut off at edges.	The viewBox doesn't match the SVG coordinates. Elements are drawn at positions outside the viewBox.	1. Check the viewBox attribute (e.g., "0 0 800 600" vs "0 0 1600 900") 2. Check element coordinates (x, y, cx, cy values) 3. If viewBox is 800x600 but elements are at x=1100, they're off-screen
Audio plays after tab switch/logout	Lecture narration continues playing after you switched to Notes tab or logged out.	The lecture's playTimer (setTimeout) is still firing advance() callbacks that call TTS.speak().	1. Check if state.stop() or state.pause() is called on tab switch (app.js switchTab) 2. Check if state._destroyed flag is set on logout 3. Check tts.js cancel() actually stops speech

5. Debugging Checklist for Non-Coders

When you see something broken on screen, follow this checklist step by step. You don't need to read code — just use your browser's built-in tools.

Step 1: Hard Refresh First (Most Common Fix)

Browsers cache (save) JavaScript files to load faster next time. When I push a fix to GitHub, your browser might still be using the OLD cached version. This is the #1 reason fixes don't seem to work.

How to hard-refresh:

- **Windows/Linux:** Press Ctrl + Shift + R or Ctrl + F5
- **Mac:** Press Cmd + Shift + R
- **Or:** Open Developer Tools (F12) → Right-click the refresh button → "Empty Cache and Hard Reload"

If this fixes it → The issue was browser caching. You're done!

Step 2: Open the Browser Console (F12)

The browser console shows error messages. When a JavaScript file fails to load or has a syntax error, the console prints a red error message. This tells you EXACTLY which file and line is broken.

How to open the console:

- **Windows/Linux:** Press F12 or Ctrl + Shift + I
- **Mac:** Press Cmd + Option + I
- **Or:** Right-click on the page → "Inspect" → click the "Console" tab

What to look for: Red text with a file name and line number. For example:

```
Uncaught SyntaxError: Unexpected token in  
data/physics/light_mirrors_lenses/chapter.js:45
```

This means chapter.js has a syntax error on line 45. The file won't load, so the chapter won't render.

Step 3: Check the Elements Tab (Inspect the SVG)

If the console has no errors but the screen is still blank, the data is loaded but the SVG elements might be invisible. You can inspect the actual SVG to check.

How to inspect the SVG:

1. Right-click on the blank chalkboard area → "Inspect"
2. The Elements panel opens with the SVG highlighted
3. Look for `<g class="el visible" data-beat="1">` — this is the visible beat
4. Inside it, look for elements with `style="opacity:0"` — these are INVISIBLE
5. If an element has `opacity:0`, it needs the chalkboard timeline to fire to set it to `opacity:1`

Quick test: In the console, type: `document.querySelector('#1b1-title').style.opacity = '1'` and press Enter. If the title appears, the issue is the timeline not firing.

Step 4: Check the viewBox vs Element Coordinates

If elements ARE visible (opacity:1) but you still see a blank screen, they might be positioned OUTSIDE the visible viewBox area. This is what happened with beat 2 of the Light chapter.

How to check:

1. In the Elements panel, find the <svg> tag
2. Look at its viewBox attribute (e.g., "0 0 800 600")
3. This means the visible area is 0 to 800 (width) and 0 to 600 (height)
4. Now look at element coordinates inside the SVG
5. If an element is at x="1100" but viewBox width is 800, it's off-screen!

The fix: Either change the viewBox to be wider (e.g., "0 0 1600 900") OR move the element's coordinates to be within the viewBox.

Step 5: Check if the Chapter File Loaded

If the chapter doesn't open at all, the chapter.js file might not have loaded. You can check this in the console.

How to check:

1. Open the console (F12)
2. Type: window.CHAPTER_DATA and press Enter
3. If it returns the chapter object → file loaded OK
4. If it returns undefined → file didn't load (check console for errors)
5. Type: window.CHAPTER_DATA.lectures.length to see how many lectures exist
6. Type: window.CHAPTER_DATA.lectures[0].beats.length to see beat count

Step 6: Quick Diagnostic Commands

Paste these into the browser console (F12) to quickly diagnose issues:

```
# Check if chapter loaded
```

```
window.CHAPTER_DATA ? "LOADED" : "NOT LOADED"
```

```
# Check lecture count
```

```
window.CHAPTER_DATA.lectures.length
```

```
# Check beat count for lecture 1
```

```
window.CHAPTER_DATA.lectures[0].beats.length
```

```
# Check SVG element count for lecture 1
```

```
document.querySelectorAll('.lec-section.active svg .el').length
```

```
# Check viewBox of active lecture
```

```
document.querySelector('.lec-section.active svg').getAttribute('viewBox')
```

```
# Check current beat
```

```
document.querySelector('.lec-counter').textContent  
# Check if beat 1 title is visible  
  
document.querySelector('#l1b1-title')?.style.opacity  
# Force beat 1 title visible (quick test)  
  
document.querySelector('#l1b1-title').style.opacity = '1'  
# Check for JavaScript errors  
  
window.onerror ? "error handler set" : "no handler"
```

6. The Light Chapter — Current State & Known Issues

The Light: Mirrors and Lenses chapter was recently rebuilt per your choreography document. Here's the current state and what to watch for.

Current Architecture

The chapter has 4 lectures, each with SVG diagrams and timeline animations:

Lecture	Beats	SVG Elements	viewBox	Status
10.1 Spherical Mirrors	10	10	1600 × 900	Fixed (beat 1 + beat 2 issues resolved)
10.2 Images in Mirrors	10	10	800 × 600	Working
10.3 Laws of Reflection	14	14	800 × 600	Working
10.4 Lenses	16	16	800 × 600	Working (was missing 4 beats, now fixed)

Recent Fixes Applied

- **Beat 1 blank screen:** The title text had `opacity:0` and depended on the chalkboard timeline to fire. If the timeline didn't fire (SVG parsing race condition), the text stayed invisible. **Fix:** Changed to `opacity:1` (visible by default). Title now appears immediately.
- **Beat 2 content off-screen:** The `viewBox` was set to 800×600 but beat 2's content uses 1600×900 coordinates (concave mirror at `x=1100`, summary labels at `y=800`). Most of the diagram was clipped. **Fix:** Reverted `viewBox` to 1600×900 and updated all background rects.
- **Beat 2 stale SVG overlay:** After exiting immersive mode, the overlay element stayed in the DOM with stale content. Re-entering immersive for a different lecture created a second overlay. **Fix:** `exitImmersive()` now removes the overlay from the DOM entirely.
- **Immersive mode frozen:** `goToBeat()` called the bare `updateUI()` function instead of `state.updateUI()` (the immersive wrapper). The immersive overlay never updated. **Fix:** Changed to `state.updateUI()` so the immersive wrapper fires.
- **Audio leak after logout:** Stale `playTimer` kept firing `TTS.speak()` after logout. **Fix:** Added `state._destroyed` flag, checked in `advance()` and `goToBeat()`.

If You Still See Issues

If you still see a blank screen after hard-refreshing (`Ctrl+Shift+R`), here's what to check:

1. **Check the console (F12)** — are there any red error messages?
2. **Check if chapter loaded:** Type `window.CHAPTER_DATA` in the console. If it returns an object, the file loaded. If undefined, the file didn't load.
3. **Check the viewBox:** Type `document.querySelector('.lec-section.active svg').getAttribute('viewBox')`. Lecture 1 should return `'0 0 1600 900'`.

4. **Check beat 1 opacity:** Type `document.querySelector('#l1b1-title')?.style.opacity`. Should return '1'.

5. **Force visible:** Type `document.querySelector('#l1b1-title').style.opacity = '1'`. If the title appears, the issue is the timeline not firing.

Summary: The 3-Layer Mental Model

Whenever something breaks, ask yourself which layer is failing:

Question	If YES →	Layer
Is the whole page blank? Does the console show errors?	The scripts didn't load. Check index.html script tags.	Loading Layer (HTML + script tags)
Is the page styled but a chapter won't open?	The chapter data file failed to load or has a syntax error. Check the console for the error.	Data Layer (chapter.js files)
Does the chapter open but the SVG diagram is blank?	The SVG elements are invisible (opacity:0) or off-screen (viewBox mismatch).	Rendering Layer (app.js + chalkboard-timeline.js + style.css)
Does the diagram show but audio plays at the wrong time?	The TTS engine or playTimer isn't being stopped properly. Check app.js goToBeat + tts.js.	Playback Layer (tts.js + app.js timers)

Remember: 90% of "it's still broken" issues are browser caching. Always hard-refresh (Ctrl+Shift+R) first before assuming the fix didn't work.